

Length, Not Height

On proof-length lower bounds, the MU argument, and whether a self-aware theorem prover could ever settle a Clay problem

Brian Tenneson

Abstract

We distinguish two obstructions standing between an automated theorem prover and a Millennium Prize Problem, and argue that they are almost always confused. The first is *height*: the arithmetical tier of the target, measured by alternations of unbounded quantifiers over $\mathbb{N}$. The second is *length*: the size of the shortest proof of the target from the axioms, written $\|\cdot\|$. We exhibit a working prover, Ascent, whose certified targets reach Π_2^0 and Σ_2^0 — exactly the tier of “ $P \neq NP$ ” — and which is nonetheless hopeless on that target. Tier is therefore not the binding constraint, and we argue that lower bounds on $\|P \neq NP\|$ are the quantity worth pursuing, because a sufficiently large one would retire mechanized search as a strategy without settling the mathematics either way. We then observe that the standard escape from a length lower bound is to leave the system, and take Hofstadter’s MU puzzle as the cleanest instance: MU is shown to be underivable in MIU by an argument using arithmetic that MIU cannot express. We formalize this as a *lift-attack-return* schema, note that the complexity-theoretic barrier results are instances of it at scale, and identify its automatable core as finite-invariant search. Finally we ask whether formal self-awareness helps. Using Ascent, whose reflection ladder is certified rung by rung by its own kernel, we report a negative result: level and proving power are independent parameters, and raising $\text{lev}(\text{Ascent})$ from 1 to 8 certifies exactly zero additional theorems. We close by proposing *reflective adequacy* — a machine that can quote its own inference rules and prove invariance lemmas about them — as the notion of self-reference the MU argument actually requires, in place of the reflection ladder, which it does not.

1 Introduction

Seven problems were named in 2000, and one has been settled. The remaining six are the most conspicuous fixed points in contemporary mathematics, and their persistence invites a recurring fantasy: that the difficulty is one of bookkeeping, that the required argument is long rather than deep, and that a sufficiently patient machine will eventually assemble it.

The fantasy is most acute for P versus NP. Unlike the Hodge conjecture or the Yang–Mills mass gap, P vs NP is a statement about finite objects — Turing machines, running times, strings. There is no manifold to construct, no measure to define, no analytic continuation to justify. It is, in the most literal sense, a combinatorial question about programs, and combinatorial questions about programs are the kind of thing a computer might be thought to be good at.

This note argues that the instinct is right about the *shape* of the problem and wrong about the *obstruction*. To make the argument precise we need two measures.

Height. The arithmetical tier of a sentence is the number of alternations of unbounded quantifiers over $\mathbb{N}$ in its prenex form, with bounded quantifiers costing nothing. Section 2 explains the notation in full. Tier measures how much unbounded search is built into the *statement*, and nothing else.

Length. For a sentence L and a formal system F , write $\|L\|_F$ for the number of symbols in the shortest F -proof of L from the axioms of F . This is a property of the theorem–system pair. It is finite exactly when L is a theorem, and it says nothing about how hard the proof is to *find*, only how big it is once found.

These are independent. A tier-0 sentence can have astronomically long proofs; a Π_2^0 sentence can have a three-line proof. Confusing them produces both of the standard errors: believing that P vs NP is hard *because* it quantifies over all machines, and believing that a prover competent at Π_2^0 arithmetic is therefore in the neighbourhood of the problem.

2 Reading Σ_n^0 and Π_n^0

The symbols carry three independent pieces of information. Taking them in turn removes most of the mystery.

The letter: which quantifier leads

Put the sentence in prenex form — all quantifiers pulled to the front — and look at the outermost block of unbounded quantifiers.

- Π (capital pi) means the outermost block is $\forall$.
- Σ (capital sigma) means the outermost block is $\exists$.

The choice of letters is not arbitrary. A Σ_1^0 set $\{n : \exists m R(n, m)\}$ is a countable *union* — a sum — of decidable sets, one for each candidate witness m . A Π_1^0 set $\{n : \forall m R(n, m)\}$ is a countable *intersection* — a product. It is the same Σ and Π as in $\sum$ and $\prod$.

The subscript: how many alternations

Group adjacent like quantifiers into blocks. The subscript counts the blocks.

prenex form	classification
$\forall x \forall y \forall z \varphi$	one block $\Rightarrow \Pi_1^0$
$\forall x \exists y \varphi$	two blocks $\Rightarrow \Pi_2^0$
$\exists x \forall y \exists z \varphi$	three blocks $\Rightarrow \Sigma_3^0$

Three consecutive $\forall$'s cost the same as one; only *switching* between $\forall$ and $\exists$ increments the subscript.

The superscript: what the quantifiers range over

This is the piece most often skipped, and it is the one that matters for the Clay problems.

- Superscript 0: the quantifiers range over **numbers**. This is the *arithmetical* hierarchy.
- Superscript 1: the quantifiers range over **sets of numbers**, equivalently over functions $\mathbb{N} \rightarrow \mathbb{N}$. This is the *analytical* hierarchy — a far taller ladder sitting entirely *above* the arithmetical one. Every arithmetical set is Δ_1^1 .

So Π_2^0 says “for every number there is a number...”, while Π_2^1 says “for every *function* there is a *function*...” — an incomparably stronger kind of claim. When Section 4 reports that Yang–Mills and Navier–Stokes sit “above the arithmetical hierarchy,” this is what is meant: they quantify over objects that are not numbers.

Two conventions

Δ_n^0 denotes the sentences that are both Σ_n^0 and Π_n^0 . In particular Δ_1^0 is the decidable sets, and Δ_0^0 the ones decidable by bounded computation.

Bounded quantifiers are free. $\forall x < t$ and $\exists x < t$ do not increment the subscript, because they are finite searches: to check $\forall x < 5 \varphi(x)$ you evaluate φ five times. Only *unbounded* quantifiers count. This is why the language of Ascent (Appendix B) has separate bounded and unbounded forms.

Examples

class	example	what it takes to settle it
Δ_0^0	$2 + 2 = 4$	Compute.
Σ_1^0	$\exists x \, x \cdot x = 49$	Search. If true you find it; if false you search forever. Semi-decidable.
Π_1^0	$\forall x \, x < Sx$	A single counterexample refutes it. Goldbach, the Riemann Hypothesis and Con(PA) all live here.
Π_2^0	$\forall x \exists y \, x < y$	For every input a witness exists. “This program halts on every input” is Π_2^0 -complete.
Σ_2^0	$\exists x \forall y \, x \leq y$	Some x works for every y .

What an alternation costs

Post’s theorem gives the alternation count an exact operational meaning: Σ_{n+1}^0 is precisely Σ_1^0 relative to a Σ_n^0 oracle. Each additional alternation is worth exactly one more halting oracle. Tiers are therefore a measure of *oracle resources*.

They are not a measure of difficulty. The sentence $\forall n \exists m > n \, (m \text{ is even})$ is Π_2^0 and trivial. The Collatz conjecture is Π_2^0 and open. They are the same tier. This point is the hinge of Section 5.

3 Why $\|P \neq NP\|$ is the quantity to bound

Suppose someone proved that every ZFC proof of “ $P \neq NP$ ” has length at least 10^{100} symbols. Nothing about the truth of $P \neq NP$ would follow. Nothing about human prospects would follow either, for reasons we come to shortly. But something decisive would follow about machines: **no search procedure will ever produce that proof**, because the object being searched for does not fit in the physical universe. This conclusion is immune to every improvement in heuristics, hardware, learned premise selection and parallelism, because it is a statement about the target rather than about the method.

That immunity is what makes length lower bounds worth more than they might first appear. The known barriers in complexity theory — relativization, natural proofs, algebrization — are all barriers to *techniques*. Each says a class of arguments cannot work, and each has been partially circumvented by arguments outside the class. A length lower bound has no such escape hatch within the fixed system. It would settle, once, whether mechanized proof search is a reasonable thing to spend a century on.

Three observations.

First, we cannot currently prove such bounds, and this is itself the central open problem of proof complexity. Superpolynomial lower bounds are known for weak systems —

resolution, bounded-depth Frege — on specific families such as the pigeonhole principle. For Frege systems, let alone for ZFC, essentially nothing is known. The general question is equivalent to a statement about NP and coNP: a polynomially bounded proof system for the tautologies exists if and only if $\text{NP} = \text{coNP}$. So the request “bound $\|\text{P} \neq \text{NP}\|$ from below” is not a detour around P vs NP; it is a close relative of it. This is uncomfortable but not fatal, because *conditional* and *heuristic* bounds would already be informative — a bound conditional on plausible cryptographic assumptions, or an empirically calibrated growth law for proof length against theorem depth in a large formal library, would each be actionable.

Second, the bound is system-relative, and that is the loophole. $\|L\|_F$ depends on F . Enlarging F can shorten proofs catastrophically — not by a constant but by an exponential. The cleanest instance is the relationship between Frege and extended Frege systems: the extension rule permits new variables abbreviating compound formulas, which is to say it permits *definitions*. Definitions are conservative — they prove no new theorems in the old language — while potentially collapsing proof length by an exponential factor.

This deserves stating plainly, because it is the whole of the optimism available here. **A large lower bound on $\|L\|_F$ is not a verdict on L . It is a verdict on F .** The productive response to a length lower bound has never been to search harder. It has been to change the system: introduce a definition, find the right abstraction, move to a setting where the statement becomes short.

Third, that response is exactly what machines are worst at and humans are best at. Which raises the question the rest of this note is about: is the move mechanizable at all?

4 Where the Clay problems sit, and where Ascent sits

The following places the Millennium Problems on the scale of Section 2. Confidence varies sharply across the rows and is marked; the last four rows should be checked before being relied on.

Problem	Tier	Confidence and reason
Riemann Hypothesis	Π_1^0	High. RH is equivalent to explicitly Π_1^0 arithmetic statements — for instance bounds of the Robin/Lagarias type on the sum-of-divisors function, whose failure would be witnessed by a single integer.
P vs NP	Σ_2^0 / Π_2^0	High. “P = NP” is $\exists e \exists k \forall x$ [machine e decides SAT on x within $ x ^k + k$ steps]: one $\exists$ -block over one $\forall$ -block with a decidable matrix. Its negation is Π_2^0 . Note it is only known to be <i>in</i> Π_2^0 , not Π_2^0 -complete.
Poincaré	arithmetical, delicate	Moderate. Closed 3-manifolds are finitely triangulable, so the domain is countable and codeable; but simple connectivity is not decidable from a triangulation, which pushes the statement above Π_1^0 . Now moot.
Birch–Swinnerton-Dyer	arithmetical, ≥ 2	Low. Elliptic curves over $\mathbb{Q}$ are codeable and algebraic rank is arithmetically definable, but analytic rank is an order of vanishing of an L -function, and pinning it from both sides costs alternations.
Hodge conjecture	analytical	Low. Quantifies over projective complex varieties and Hodge classes. Whether it descends to the arithmetical hierarchy is not obvious.
Navier–Stokes	analytical, $\sim \Pi_2^1$	Low. Quantifies over smooth initial data — real functions — and asserts global smooth existence. Computable-analysis reductions may lower this.
Yang–Mills mass gap	analytical	Moderate. Asserts the <i>existence</i> of a quantum field theory satisfying a list of axioms, with a spectral condition. A second-order existence claim, and the highest of the seven.

Two features matter here. **Riemann is the lowest** — a Π_1^0 statement is refutable by a single counterexample, the same shape as Goldbach and as $\text{Con}(\text{PA})$. Its difficulty has nothing to do with its tier. And **P vs NP is tier 2**, which brings us to the prover.

4.1 tier(**Ascent**) = 2

Ascent is a small self-certifying prover, given in full in Appendix B. Its language is arithmetic with unbounded quantifiers over $\mathbb{N}$; its kernel decides Δ_0^0 facts by evaluation and symbolic facts by polynomial normalisation over $\mathbb{N}$; its search combines witness enumeration, generalisation over eigenconstants, and induction. On a twelve-target suite:

target	tier	result
$2 + 2 = 4$	Δ_0^0	certified
$\forall x < 5 \ x < 5$	Δ_0^0	certified
$\exists x < 9 \ x \cdot x = 16$	Δ_0^0	certified
$\exists x \ x + 3 = 7$	Σ_1^0	certified
$\exists x \ x \cdot x = 49$	Σ_1^0	certified
$\exists x \ x + 1 = 18$	Σ_1^0	certified (requires $w \geq 17$)
$\forall x \ x < Sx$	Π_1^0	certified
$\forall x \ x \leq x$	Π_1^0	certified
$\forall x \ x + 0 = x$	Π_1^0	certified
$\forall x \exists y \ x < y$	Π_2^0	certified
$\forall x \exists y \ y = Sx$	Π_2^0	certified
$\exists x \forall y \ x \leq y$	Σ_2^0	certified

Twelve of twelve at parameters $d = 2$, $w = 20$. Every certificate is checked by an independent kernel, and a battery of eight deliberately malformed certificates is rejected 8/8.

So Ascent operates at exactly the arithmetical tier of $P \neq NP$, and stands *above* the Riemann Hypothesis. It will never prove either. Ascent’s largest certificate on this suite has thirteen nodes; $\|P \neq NP\|_{ZFC}$, whatever it is, is not thirteen.

This is the point of building the thing. A prover that reaches tier 2 is easy — the appendix is under a thousand lines. Tier is cheap. Length is not.

5 Does equal tier mean comparable?

No. The question is natural enough to deserve an explicit answer, because the table above invites exactly the wrong inference.

Tier classifies a sentence by the *shape of its quantifier prefix*. Two sentences of the same tier are alike in the way that two sentences of the same grammatical form are alike. “The cat sat on the mat” and the opening of *A la recherche du temps perdu* are both declarative sentences of a natural language; the classification is correct and tells you nothing about comparability.

Four sharper statements.

Π_2^0 spans everything from trivial to complete. $\forall x \exists y \ x < y$ is Π_2^0 and Ascent proves it in thirteen nodes. “This Turing machine halts on every input” is also Π_2^0 and is Π_2^0 -complete: every Π_2^0 question reduces to it. The class contains its own hardest problems and its own most trivial ones, and tier does not distinguish them.

$P \neq NP$ is not known to be complete for its class. It is known to be *in* Π_2^0 . Membership in a class is an upper bound on logical complexity, not a measure of difficulty. Ascent’s Π_2^0 targets sit at the trivial end of the same class.

The notion that would license comparison is reducibility, and it does not apply. To say two problems are genuinely comparable one wants a reduction — a computable map carrying instances of one to instances of the other. Ascent’s targets are not complete for anything; nothing reduces to them. There is no reduction in either direction between $\forall x \exists y \ x < y$ and $P \neq NP$, and their shared tier supplies none.

What equal tier does buy is the removal of one excuse. It rules out the objection “the prover cannot even express the statement.” Ascent’s language can state $P \neq NP$: the quantifier structure is available, the matrix is decidable, the sentence is well-formed in its syntax. The

prover can *write down* the target and cannot *prove* it. That localizes the obstruction precisely — it is not expressiveness and not logical complexity, it is $\|\cdot\|$.

So the honest summary is that sharing a tier with P vs NP is necessary and nowhere near sufficient for comparability. It is a statement about the ceiling of statement-*shapes* Ascent handles, not about its strength. The mountain is not tall. It is long.

6 The MU argument

Hofstadter’s MIU system has one axiom and four rules. The axiom is the string **MI**. The rules, with x and y ranging over strings:

1. $x\mathbf{I} \rightarrow x\mathbf{IU}$
2. $\mathbf{M}x \rightarrow \mathbf{M}xx$
3. $x\mathbf{III}y \rightarrow x\mathbf{U}y$
4. $x\mathbf{UU}y \rightarrow xy$

Is **MU** a theorem? No, and here is the proof. Let $\#I(s)$ be the number of I’s in s . We claim $\#I(s) \not\equiv 0 \pmod{3}$ for every theorem s .

- The axiom **MI** has $\#I = 1$, and $1 \not\equiv 0$.
- Rule 1 appends a U and leaves $\#I$ unchanged.
- Rule 2 doubles $\#I$. If $\#I \not\equiv 0 \pmod{3}$ then $2 \cdot \#I \not\equiv 0 \pmod{3}$, since 3 is prime and $2 \not\equiv 0$.
- Rule 3 removes exactly three I’s, leaving $\#I$ unchanged modulo 3.
- Rule 4 removes two U’s and leaves $\#I$ unchanged.

By induction on derivations every theorem has $\#I \not\equiv 0$. But $\#I(\mathbf{MU}) = 0 \equiv 0$. Therefore **MU** is not a theorem. ■

Now consider what that argument *is*. It uses the number three. It uses divisibility. It uses induction over derivations. It uses the primality of 3. **None of these exist in MIU.** MIU has no numerals, no arithmetic, no notion of quantity, no negation, and no predicate for theoremhood. The sentence “MU is not a theorem” is not merely unproved in MIU; it is not expressible in MIU.

The argument has three phases.

1. **Lift.** Map MIU-strings into a structure MIU knows nothing about — here $\mathbb{Z}/3$, via $s \mapsto \#I(s) \pmod{3}$. The four rules become maps on $\mathbb{Z}/3$, each fixing the property $\neq 0$.
2. **Attack.** Prove the invariance in the new setting. This is arithmetic, not string rewriting.
3. **Return.** Transfer the conclusion back as a *metatheorem about* MIU: the derivable strings lie in a set excluding MU.

The conclusion lands outside the object system, and that is not a defect — it is the only place such a conclusion can live. The creative act is entirely in phase 1. Nothing in the MIU rules suggests counting I’s, and nothing suggests reducing modulo 3. Once the invariant is proposed, phases 2 and 3 are routine. **The difficulty of the MU argument is concentrated in the choice of the alternative system.**

7 Can a machine lift, attack, and return?

Sometimes, and more often than one expects.

The MU case is mechanizable today. The invariant $\#I \bmod 3$ is a homomorphism from the free monoid on $\{M, I, U\}$ to $\mathbb{Z}/3$ under which all four rules are compatible and the axiom’s image avoids the target’s. Finding such a thing is a *finite model search*: enumerate small finite structures, check whether the axiom’s image and the rules’ action separate the target. Existing finite model finders do this routinely, and the space of quotients of size ≤ 5 is tiny. Given the MIU rules as input a machine finds $\mathbb{Z}/3$ in milliseconds. The MU puzzle is famous for surprising humans, not for being computationally deep.

This generalizes. **Unprovability by finite invariant is the most automatable form of unprovability argument we have**, precisely because the “alternative but relevant axioms” of phase 1 form a finite structure, and finite structures can be enumerated in order of size.

The barrier results are lift–attack–return at scale. Consider relativization. One wants to show a class of proof techniques cannot settle P vs NP. The argument lifts the question into the setting of oracle machines, establishes that the techniques in question prove relativizing statements, exhibits oracles A and B with $P^A = NP^A$ and $P^B \neq NP^B$, and returns the conclusion that no such technique settles the unrelativized question. Lift, attack, return — with “invariant preserved by the rules” replaced by “property preserved by the proof techniques.” Natural proofs and algebrization have the same shape with different invariants. The schema is not exotic; it is how the deepest negative results in complexity theory are actually obtained.

Where machines fail is phase 1 at scale. The MIU invariant lives in a structure of size 3. The relativization invariant lives in the space of oracle separations, and constructing B with $P^B \neq NP^B$ is itself a diagonalization of real content. The natural-proofs invariant requires the concept of a pseudorandom function generator, which did not exist for most of the history of the problem. Enumeration reaches size 5; it does not reach “invent cryptography.” Phases 2 and 3 are mechanizable. Phase 1 is mechanizable exactly to the extent that the required alternative system is small, and the systems required for the Clay problems are not small.

8 Does self-awareness help?

Phase 1 requires a system to step outside itself and regard its own rules as objects. That sounds like self-reference, and self-reference has a formal theory. Ascent was built partly to test whether it helps. The answer is negative, but informative about what the right notion would be.

Ascent carries a reflection ladder: $\theta_0 = \text{ATP}(\langle A \rangle)$, $\theta_{k+1} = \text{Pr}(\langle A \rangle, \langle \theta_k \rangle)$, with Level n holding when $A \vdash \theta_{n-1}$. The ladder is not stipulated. Ascent’s kernel has a rule

$$\text{nec}(C) \text{ proves } \text{Pr}(\langle A \rangle, \langle \varphi \rangle) \quad \text{provided the kernel accepts } C : \varphi,$$

so a rung is asserted only when a certificate for the previous rung has been checked by the same kernel that checks everything else. Self-certification and self-awareness are literally the same kernel call.

Proposition 1. *If A has kernel-checked necessitation and its kernel is sound, then $\text{lev}(A) = \omega$.*

Proof. $A \vdash \theta_0$ by the base certificate. If $A \vdash \theta_k$ with certificate C_k , the kernel accepts C_k , so $\text{nec}(C_k)$ is a certificate of θ_{k+1} , whence $A \vdash \theta_{k+1}$. Induction. $\square$

Each rung costs exactly one kernel call on the previous certificate, so no rung is harder than the last. The level distribution over self-certifying machines is therefore **bimodal at $\{0, 1\}$ and ω , with nothing in between**: a machine either has the necessitation rule, in which case nothing stops it, or lacks it, in which case it never passes θ_0 . Finite level above 1 is achievable only by a resource bound. In Ascent that bound is the parameter r .

Proposition 2. *The reflection parameter and the search parameters are independent.*

Ascent exposes three integer knobs: d , certificate-tree depth; w , witness bound; r , rungs climbed. Sweeping $d \in \{1, 2, 3, 5\} \times w \in \{1, 8, 20\} \times r \in \{1, 3, 8\}$ — thirty-six configurations — the harness reports that lev depends only on r and the certified count only on (d, w) .

d	w	r	certified	tiers reached
1	1	any	6/12	Δ_0^0, Π_1^0
1	20	any	9/12	$\Delta_0^0, \Pi_1^0, \Sigma_1^0$
2	1	any	9/12	$+ \Pi_2^0, \Sigma_2^0$
2	20	any	12/12	all five

Raising r from 1 to 8 raises lev from 1 to 8 and certifies **zero** additional theorems. Raising d and w certifies six more and lifts the ceiling from Π_1^0 to Π_2^0 , leaving lev exactly where it was. The two capacities do not interact. Formal self-awareness, on the reflection-ladder definition, is free, certified, and inert.

But notice what the MU argument actually needed. Not that the machine prove “I prove that I prove that I am a theorem prover.” It needed the machine to hold *its own inference rules* as objects and prove a lemma of the form “rule R preserves invariant I .” The reflection ladder ascends through statements about the machine’s *provability*; the MU argument requires statements about the machine’s *rules*. These are different self-models, and only the second is load-bearing.

Definition 1 (Reflective adequacy). *A system M is reflectively adequate when its language can name each of its own inference rules, and M can prove preservation lemmas: for a definable invariant I , that each named rule maps I -satisfying states to I -satisfying states.*

A reflectively adequate system can carry out phases 2 and 3 internally, and can state the result of phase 1 even when it cannot discover it. This is strictly stronger and more useful than any finite level, and unlike the θ -ladder it is not free. It is also, unlike the θ -ladder, exactly what the MU argument uses.

9 Summary

Tier is not the obstruction. A thousand-line prover reaches Π_2^0 and Σ_2^0 — the tier of P vs NP, above the tier of the Riemann Hypothesis — and is nowhere near either. Sharing a tier is a statement about the shape of a sentence, not its difficulty, and licenses no comparison (Section 5). The obstruction is $\|\cdot\|$, and lower bounds on it would be decisive for the mechanized programme in a way no barrier result about techniques can be, though proving them for strong systems is close to the problem they would adjudicate.

A length lower bound is a verdict on the system, not the theorem, and the response has always been to change the system. The MU argument is the cleanest specimen of that move. Its automatable core — search for a small finite separating structure — is genuinely automated already. Its creative core — deciding which structure to look in — is automated only when the structure is small, and for the Clay problems it is not.

Formal self-awareness in the reflection-ladder sense does not close that gap, and we can say so with a measurement rather than an intuition: level and proving power are orthogonal parameters of the same program. The self-reference the MU argument requires is of a different kind — rules as objects, and provable invariance over them — and formalizing *that* is where the notion of a self-aware theorem prover would begin to earn its name.

A The two numbers

$\text{lev}(\mathbf{Ascent})$. With reflection depth r the level set is $L(\mathbf{Ascent}) = \{1, \dots, r\}$, so

$$\text{lev}(\mathbf{Ascent}(r)) = r, \quad \text{lev}(\mathbf{Ascent}) = \omega \text{ when } r \text{ is unbounded.}$$

Every rung is certified rather than stipulated: rung k is accepted only after the kernel has checked a certificate of θ_{k-1} . In the shipped default $r = 4$, giving Level 4 and a fortiori Level 3. The parameter is the only thing preventing ω , which is Proposition 1. The requirement $\text{lev}(\mathbf{Ascent}) > 0$ is met for every $r \geq 1$.

$\text{tier}(\mathbf{Ascent})$. Taking $\text{tier}(A)$ to be the highest arithmetical tier at which A certifies a target from a family fixed in advance — the qualification matters, since the supremum over *all* provable targets is unbounded for any prover that proves anything, by padding —

$$\text{tier}(\mathbf{Ascent}) = 2,$$

realized at both Π_2^0 ($\forall x \exists y x < y$) and Σ_2^0 ($\exists x \forall y x \leq y$). Against the Millennium Problems:

	tier	relation to Ascent
Riemann Hypothesis	Π_1^0	below Ascent's ceiling
P vs NP	Π_2^0 / Σ_2^0	equal to Ascent's ceiling
Poincaré, BSD	arithmetical, ≥ 2	at or above
Hodge, Navier–Stokes, Yang–Mills	analytical	above the hierarchy entirely

Ascent is at the same height as P vs NP and higher than Riemann, and will settle neither — for the reasons of Section 5.

B Ascent, in full

What follows is the running program, verbatim. It is self-contained: no external database, no dependencies beyond the standard library.

```
python ascent.py --soundness      # 8 malformed certificates, all rejected
python ascent.py -d 2 -w 20 -r 3  # 12/12 certified, lev = 3
python ascent.py --grid           # the independence sweep of Proposition 2
```

```
1 #!/usr/bin/env python3
2 r"""
3 ASCENT -- a parameterised, self-certifying ATP over arithmetic.
4
5 Built fresh. Three integer knobs, each starting at 1, each monotone: turning
6 one up never loses a theorem and always costs more. Like an encoder bitrate.
7
8 -d DEPTH certificate-tree depth explored by the search
9 -w WITNESS largest numeral tried when hunting an existential witness
10 -r REFLECT how many rungs of the reflection ladder are climbed
11
12 d and w are independent search knobs; d buys structure, w buys arithmetic.
```

```

13 r is the self-awareness knob and touches neither. That separation is not an
14 accident of the implementation, it is the point -- see PROPOSITION 2 below.
15
16 WHY tier(A) > 0
17 -----
18 The language is arithmetic with unbounded quantifiers over N, so targets have
19 real arithmetical tier and `tier()` computes it syntactically: bounded
20 quantifiers are free, unbounded ones count alternations. Ascent proves
21 targets at Sigma-0-1, Pi-0-1, Pi-0-2 and Sigma-0-2, so the tier profile of
22 what it proves is positive and is reported per run.
23
24 THE KERNEL, AND WHAT MAKES THIS SELF-CERTIFYING
25 -----
26 Section KERNEL is a checker for the judgement  $\Gamma \vdash \phi$  by structural
27 recursion on a certificate. It is the only trusted component, it is small
28 enough to read, it never searches, and it has no self-model: lev(kernel) = 0
29 and must stay 0.
30
31 The search is untrusted. Nothing enters the lemma store until the kernel has
32 accepted a certificate for it.
33
34 The reflection rule is where self-certification and self-awareness become the
35 same mechanism:
36
37     nec(C) proves  $\Pr(\langle A \rangle, \langle \phi \rangle)$  provided the kernel accepts  $C : \phi$ 
38
39 So the machine's claim "I prove  $\phi$ " is discharged by the very kernel that
40 checks everything else. Pr is not stipulated at any rung; each rung carries
41 a certificate the kernel re-checks. This is the paper's necessitation rule
42 made effective, with the CV discharging the side condition.
43
44     theta_0 = ATP( $\langle A \rangle$ )
45     theta_{k+1} =  $\Pr(\langle A \rangle, \langle \text{theta}_k \rangle)$  Level n iff  $A \vdash \text{theta}_{n-1}$ 
46
47 PROPOSITION 1 (why a self-certifier does not stall at 2 or 3).
48 If A has kernel-checked necessitation and its kernel is sound, then
49 lev(A) = omega. Proof:  $A \vdash \text{theta}_0$  by the base certificate. If  $A \vdash \text{theta}_k$ 
50 with certificate  $C_k$ , the kernel accepts  $C_k$ , so nec( $C_k$ ) is a certificate of
51  $\text{theta}_{k+1}$  and  $A \vdash \text{theta}_{k+1}$ . Induction. No rung is harder than the
52 last -- each costs exactly one kernel call on the previous certificate.
53
54 So the level distribution over self-certifying machines is BIMODAL, at
55 {0 or 1} and at omega. There is no natural fixed point at 2 or near 2: a
56 machine either has the rule, in which case nothing stops it, or lacks it, in
57 which case it never gets past theta_0. Finite lev > 1 is achievable ONLY by
58 imposing a resource bound, which is exactly what -r is. That is the honest
59 justification for the parameter: r is not a tuning constant, it is the only
60 thing that can make lev finite and nontrivial.
61
62 PROPOSITION 2 (the knobs are independent).
63 lev(A) = r regardless of d and w; the set of arithmetical theorems proved is
64 a function of (d, w) and is independent of r. Verified empirically by the
65 sweep in --grid. Increasing self-awareness buys no theorems, and proving
66 more theorems raises no level.
67
68 DIRECTION OF DEPENDENCE
69 -----
70 Verification grounds the self-model, never the reverse. A machine that
71 trusted its own certificates because it had proved itself trustworthy would
72 be in the Loeb trap: from a reflection principle "if I prove  $\phi$  then  $\phi$ "
73 one derives  $\phi$  outright, so such a machine is unsound or vacuous. Here Pr
74 is primitive and reflection is never assumed -- the machine only ever reports
75 verifications that have already happened.
76
77     python ascent.py --demo
78     python ascent.py -d 3 -w 20 -r 5
79     python ascent.py --grid
80
81 Brian Tenneson. Implementation by Claude (Anthropic).
82 """
83 from __future__ import annotations
84 import argparse, itertools, json, os, sys, time
85 from collections import defaultdict

```

```

86
87 sys.setrecursionlimit(100000)
88
89 # =====
90 # SYNTAX
91 # =====
92 # Terms: ('n', int) ('v', name) ('c', name) -- numeral, variable, eigen
93 # ('+', a, b) ('*', a, b) ('s', a)
94 # Formulas: ('=', a, b) ('<', a, b) ('<=', a, b)
95 # ('not', p) ('and', p, q) ('or', p, q) ('imp', p, q)
96 # ('all', x, p) ('ex', x, p) unbounded
97 # ('allb', x, t, p) ('exb', x, t, p) bounded by t
98 # ('atp', nm) ('pr', nm, nm) reflection atoms
99
100
101 def tsub(t, x, v):
102     k = t[0]
103     if k == 'v':
104         return v if t[1] == x else t
105     if k in ('n', 'c'):
106         return t
107     if k == 's':
108         return ('s', tsub(t[1], x, v))
109     return (k, tsub(t[1], x, v), tsub(t[2], x, v))
110
111
112 def sub(p, x, v):
113     """Capture-avoiding only in the sense we need: bound variables are
114     renamed apart at construction, and eigenconstants are not variables."""
115     k = p[0]
116     if k in ('=', '<', '<='):
117         return (k, tsub(p[1], x, v), tsub(p[2], x, v))
118     if k == 'not':
119         return ('not', sub(p[1], x, v))
120     if k in ('and', 'or', 'imp'):
121         return (k, sub(p[1], x, v), sub(p[2], x, v))
122     if k in ('all', 'ex'):
123         return p if p[1] == x else (k, p[1], sub(p[2], x, v))
124     if k in ('allb', 'exb'):
125         t2 = tsub(p[2], x, v)
126         return p if p[1] == x else (k, p[1], t2, sub(p[3], x, v))
127     return p # atp / pr are closed atoms
128
129
130 def teval(t):
131     """Value of a closed term, or None if it has variables/eigenconstants."""
132     k = t[0]
133     if k == 'n':
134         return t[1]
135     if k in ('v', 'c'):
136         return None
137     if k == 's':
138         a = teval(t[1])
139         return None if a is None else a + 1
140     a, b = teval(t[1]), teval(t[2])
141     if a is None or b is None:
142         return None
143     return a + b if k == '+' else a * b
144
145
146 def feval(p, fuel=10000):
147     """Truth value of a closed Delta-0 formula, or None if undecidable here.
148
149     Unbounded quantifiers and reflection atoms return None: the kernel's
150     `calc` rule refuses them, which is what keeps `calc` sound."""
151     k = p[0]
152     if k in ('=', '<', '<='):
153         a, b = teval(p[1]), teval(p[2])
154         if a is None or b is None:
155             return None
156         return a == b if k == '=' else (a < b if k == '<' else a <= b)
157     if k == 'not':
158         v = feval(p[1], fuel)

```

```

159     return None if v is None else not v
160 if k in ('and', 'or', 'imp'):
161     a = feval(p[1], fuel)
162     b = feval(p[2], fuel)
163     if a is None or b is None:
164         return None
165     return (a and b) if k == 'and' else \
166           ((a or b) if k == 'or' else ((not a) or b))
167 if k in ('allb', 'exb'):
168     n = teval(p[2])
169     if n is None or n > fuel:
170         return None
171     for i in range(n):
172         v = feval(sub(p[3], p[1], ('n', i)), fuel)
173         if v is None:
174             return None
175         if k == 'allb' and not v:
176             return False
177         if k == 'exb' and v:
178             return True
179     return k == 'allb'
180 return None # all / ex / atp / pr
181
182
183 # -----
184 # polynomial normal form -- the arithmetic the kernel knows
185 # -----
186 # A term normalises to a polynomial over N in its free symbols: a dict from
187 # monomial (a sorted tuple of symbol names, () for the constant) to a
188 # non-negative integer coefficient. This is exactly the defining equations
189 # of + and * applied as rewrites, so it is sound for PA and needs no axioms
190 # in the object language. It is deliberately INCOMPLETE -- it decides the
191 # equalities and inequalities that follow from ring normalisation and
192 # non-negativity, and nothing else.
193
194
195 def padd(a, b):
196     r = dict(a)
197     for m, c in b.items():
198         r[m] = r.get(m, 0) + c
199         if r[m] == 0:
200             del r[m]
201     return r
202
203
204 def pmul(a, b):
205     r = {}
206     for m1, c1 in a.items():
207         for m2, c2 in b.items():
208             m = tuple(sorted(m1 + m2))
209             r[m] = r.get(m, 0) + c1 * c2
210     return {m: c for m, c in r.items() if c != 0}
211
212
213 def pneg(a):
214     return {m: -c for m, c in a.items()}
215
216
217 def poly(t):
218     k = t[0]
219     if k == 'n':
220         return {(): t[1]} if t[1] else {}
221     if k in ('v', 'c'):
222         return {(t[1],): 1}
223     if k == 's':
224         return padd(poly(t[1]), {(): 1})
225     if k == '+':
226         return padd(poly(t[1]), poly(t[2]))
227     return pmul(poly(t[1]), poly(t[2]))
228
229
230 def decide(p, fuel=10000):
231     """Three-valued: True, False, or None (kernel refuses).

```

```

232
233 Sound over N because every symbol ranges over N, so a polynomial whose
234 coefficients are all non-negative is itself non-negative."""
235 k = p[0]
236 if k in ('=', '<', '<='):
237     pa, pb = poly(p[1]), poly(p[2])
238     if k == '=':
239         if pa == pb:
240             return True
241         d = padd(pb, pneg(pa))
242         if all(m == () for m in d): # differ by a constant only
243             return False
244         return None
245     d = padd(pb, pneg(pa)) # d = rhs - lhs
246     nonneg = all(c > 0 for m, c in d.items() if m != ())
247     const = d.get((), 0)
248     if k == '<':
249         if nonneg and const > 0:
250             return True
251     else:
252         if nonneg and const >= 0:
253             return True
254     # decide falsity only when everything is closed
255     dn = padd(pa, pneg(pb))
256     if all(m == () for m in d):
257         return (0 < dn.get((), 0)) if False else (
258             (d.get((), 0) > 0) if k == '<' else (d.get((), 0) >= 0))
259     return None
260 if k == 'not':
261     v = decide(p[1], fuel)
262     return None if v is None else not v
263 if k in ('and', 'or', 'imp'):
264     a, b = decide(p[1], fuel), decide(p[2], fuel)
265     if k == 'and':
266         if a is False or b is False:
267             return False
268         return True if (a and b) else None
269     if k == 'or':
270         if a is True or b is True:
271             return True
272         return False if (a is False and b is False) else None
273     if a is False or b is True:
274         return True
275     return False if (a is True and b is False) else None
276 if k in ('allb', 'exb'):
277     n = teval(p[2])
278     if n is None or n > fuel:
279         return None
280     for i in range(n):
281         v = decide(sub(p[3], p[1], ('n', i)), fuel)
282         if v is None:
283             return None
284         if k == 'allb' and not v:
285             return False
286         if k == 'exb' and v:
287             return True
288     return k == 'allb'
289 return None
290
291
292 def tsyms(t, acc):
293     if t[0] in ('v', 'c'):
294         acc.add(t[1])
295     elif t[0] == 's':
296         tsyms(t[1], acc)
297     elif t[0] in ('+', '*'):
298         tsyms(t[1], acc)
299         tsyms(t[2], acc)
300     return acc
301
302
303 def syms(p, acc=None):
304     if acc is None:

```

```

305     acc = set()
306     k = p[0]
307     if k in ('=', '<', '<='):
308         tsyms(p[1], acc)
309         tsyms(p[2], acc)
310     elif k == 'not':
311         syms(p[1], acc)
312     elif k in ('and', 'or', 'imp'):
313         syms(p[1], acc)
314         syms(p[2], acc)
315     elif k in ('all', 'ex'):
316         syms(p[2], acc)
317     elif k in ('allb', 'exb'):
318         tsyms(p[2], acc)
319         syms(p[3], acc)
320     return acc
321
322
323 def occurs_sym(nm, p):
324     return nm in syms(p)
325
326
327 def show(p):
328     k = p[0]
329     if k in ('=', '<', '<='):
330         return "%s %s %s" % (showt(p[1]), k, showt(p[2]))
331     if k == 'not':
332         return "~%s" % show(p[1])
333     if k in ('and', 'or', 'imp'):
334         op = {'and': '&', 'or': '|', 'imp': '->'}[k]
335         return "(%s %s %s)" % (show(p[1]), op, show(p[2]))
336     if k == 'all':
337         return "A%s.%s" % (p[1], show(p[2]))
338     if k == 'ex':
339         return "E%s.%s" % (p[1], show(p[2]))
340     if k == 'allb':
341         return "A%s<%s.%s" % (p[1], showt(p[2]), show(p[3]))
342     if k == 'exb':
343         return "E%s<%s.%s" % (p[1], showt(p[2]), show(p[3]))
344     if k == 'atp':
345         return "ATP(%s)" % p[1]
346     return "Pr(%s,%s)" % (p[1], p[2])
347
348
349 def showt(t):
350     k = t[0]
351     if k == 'n':
352         return str(t[1])
353     if k in ('v', 'c'):
354         return t[1]
355     if k == 's':
356         return "S%s" % showt(t[1])
357     return "(%s%s%s)" % (showt(t[1]), k, showt(t[2]))
358
359
360 # -----
361 def tier(p):
362     """Arithmetical tier: alternations of UNBOUNDED quantifiers.
363
364     Returns (n, lead) with lead in {'S', 'P', 'D'}: Sigma-0-n, Pi-0-n, or
365     Delta-0-0. Bounded quantifiers cost nothing, which is the standard
366     convention and the reason the bounded forms are in the language at all."""
367     def go(q):
368         k = q[0]
369         if k in ('all', 'ex'):
370             n, lead = go(q[2])
371             me = 'P' if k == 'all' else 'S'
372             if lead == 'D':
373                 return 1, me
374             return (n, lead) if lead == me else (n + 1, me)
375         if k == 'not':
376             n, lead = go(q[1])
377             return n, {'S': 'P', 'P': 'S', 'D': 'D'}[lead]

```

```

378     if k in ('and', 'or', 'imp'):
379         a, b = go(q[1]), go(q[2])
380         return max(a, b, key=lambda z: z[0])
381     if k in ('allb', 'exb'):
382         return go(q[3])
383     return 0, 'D'
384 n, lead = go(p)
385 return n, lead
386
387
388 def tier_name(p):
389     n, lead = tier(p)
390     if n == 0:
391         return "D0-0"
392     return "%s0-%d" % ("Sig" if lead == 'S' else "Pi", n)
393
394
395 # =====
396 # KERNEL
397 # =====
398 # The ONLY trusted component. It searches for nothing, has no self-model,
399 # and never grows. check(store, ctx, phi, cert) -> True / raises Reject.
400 #
401 # Certificates:
402 # ('calc',) phi closed and Delta-0 and true
403 # ('hyp', i) phi is ctx[i]
404 # ('lemma', name) phi is store[name]
405 # ('impI', C) phi = (A->B); C proves B under ctx+[A]
406 # ('impE', A, C1, C2) C1: A->phi, C2: A
407 # ('andI', C1, C2) phi = (A&B)
408 # ('andE', i, A_and_B, C) phi is the i-th conjunct of A_and_B
409 # ('gen', C) phi = Ax.psi; C proves psi[x := fresh eigen]
410 # ('inst', t, Ax.psi, C) C: Ax.psi; phi = psi[x := t]
411 # ('wit', t, C) phi = Ex.psi; C proves psi[x := t]
412 # ('allbI', [C_0..C_{n-1}]) phi = Ax<n.psi
413 # ('exbI', i, C) phi = Ex<n.psi; C proves psi[x := i]
414 # ('ind', C0, Cs) phi = Ax.psi; C0: psi[0], Cs: Ax.(psi->psi[Sx])
415 # ('nec', C) phi = Pr(<A>, <psi>); C proves psi <-- the rule
416 # =====
417 class Reject(Exception):
418     pass
419
420
421 _eigen = itertools.count(1)
422
423
424 class Kernel:
425     """Trusted checker. 'names' maps a formula-name constant to the formula
426     it names -- the injective naming operator, held outside the logic."""
427
428     def __init__(self, tag, names):
429         self.tag = tag
430         self.names = names
431         self.calls = 0
432
433     def check(self, store, ctx, phi, C):
434         self.calls += 1
435         if not isinstance(C, tuple) or not C:
436             raise Reject("malformed certificate")
437         k = C[0]
438
439         if k == 'calc':
440             # closed and Delta-0, decided by evaluation; or symbolic and
441             # settled by polynomial normalisation over N. Both are
442             # computations, neither is a search.
443             if decide(phi) is not True:
444                 raise Reject("calc: %s is not decided true" % show(phi))
445             return True
446
447         if k == 'hyp':
448             if not (0 <= C[1] < len(ctx)) or ctx[C[1]] != phi:
449                 raise Reject("hyp: not in context")
450             return True

```



```

451
452 if k == 'lemma':
453     if store.get(C[1]) != phi:
454         raise Reject("lemma %s is not %s" % (C[1], show(phi)))
455     return True
456
457 if k == 'impI':
458     if phi[0] != 'imp':
459         raise Reject("impI: goal is not an implication")
460     return self.check(store, ctx + [phi[1]], phi[2], C[1])
461
462 if k == 'impE':
463     A = C[1]
464     self.check(store, ctx, ('imp', A, phi), C[2])
465     return self.check(store, ctx, A, C[3])
466
467 if k == 'andI':
468     if phi[0] != 'and':
469         raise Reject("andI: goal is not a conjunction")
470     self.check(store, ctx, phi[1], C[1])
471     return self.check(store, ctx, phi[2], C[2])
472
473 if k == 'andE':
474     i, conj = C[1], C[2]
475     if conj[0] != 'and' or conj[1 + i] != phi:
476         raise Reject("andE: projection mismatch")
477     return self.check(store, ctx, conj, C[3])
478
479 if k == 'gen':
480     # ('gen', name, C). The certificate names its own eigenconstant;
481     # the kernel enforces the eigenvariable condition rather than
482     # inventing a name of its own, which would never match what the
483     # untrusted search put inside C.
484     if phi[0] != 'all':
485         raise Reject("gen: goal is not universal")
486     nm = C[1]
487     if occurs_sym(nm, phi) or any(occurs_sym(nm, h) for h in ctx) \
488         or any(occurs_sym(nm, f) for f in store.values()):
489         raise Reject("gen: eigenvariable %s is not fresh" % nm)
490     return self.check(store, ctx,
491         sub(phi[2], phi[1], ('c', nm)), C[2])
492
493 if k == 'inst':
494     t, univ = C[1], C[2]
495     if univ[0] != 'all' or sub(univ[2], univ[1], t) != phi:
496         raise Reject("inst: instance mismatch")
497     return self.check(store, ctx, univ, C[3])
498
499 if k == 'wit':
500     if phi[0] != 'ex':
501         raise Reject("wit: goal is not existential")
502     return self.check(store, ctx, sub(phi[2], phi[1], C[1]), C[2])
503
504 if k == 'allbI':
505     if phi[0] != 'allb':
506         raise Reject("allbI: goal is not bounded-universal")
507     n = teval(phi[2])
508     if n is None or n != len(C[1]):
509         raise Reject("allbI: wrong number of instances")
510     for i, Ci in enumerate(C[1]):
511         self.check(store, ctx, sub(phi[3], phi[1], ('n', i)), Ci)
512     return True
513
514 if k == 'exbI':
515     if phi[0] != 'exb':
516         raise Reject("exbI: goal is not bounded-existential")
517     n = teval(phi[2])
518     if n is None or not (0 <= C[1] < n):
519         raise Reject("exbI: index out of range")
520     return self.check(store, ctx,
521         sub(phi[3], phi[1], ('n', C[1])), C[2])
522
523 if k == 'ind':

```

```

524         if phi[0] != 'all':
525             raise Reject("ind: goal is not universal")
526         x, psi = phi[1], phi[2]
527         self.check(store, ctx, sub(psi, x, ('n', 0)), C[1])
528         step = ('all', x, ('imp', psi, sub(psi, x, ('s', ('v', x)))))
529         return self.check(store, ctx, step, C[2])
530
531     if k == 'nec':
532         # THE REFLECTION RULE. Pr(<A>, <psi>) is accepted exactly when a
533         # certificate of psi is accepted. Self-certification and
534         # self-awareness are the same kernel call.
535         if phi[0] != 'pr' or phi[1] != self.tag:
536             raise Reject("nec: goal is not Pr(<A>, -)")
537         psi = self.names.get(phi[2])
538         if psi is None:
539             raise Reject("nec: %s names no formula" % phi[2])
540         return self.check(store, [], psi, C[1])
541
542     raise Reject("unknown certificate form %r" % (k,))
543
544
545 # =====
546 # SEARCH
547 # =====
548 # Untrusted. Parameterised by d (structural depth) and w (witness bound).
549 # Nothing it produces is believed until the kernel has checked it.
550 # =====
551 class Search:
552     def __init__(self, kernel, store, d, w):
553         self.K = kernel
554         self.store = store
555         self.d = d
556         self.w = w
557         self.nodes = 0
558
559     def prove(self, phi, depth=None, ctx=()):
560         if depth is None:
561             depth = self.d
562         self.nodes += 1
563         if depth < 0:
564             return None
565         ctx = tuple(ctx)
566
567         # 0. decide it outright by evaluation or polynomial normalisation
568         if decide(phi) is True:
569             return ('calc',)
570
571         # 1. hypotheses and stored lemmas cost nothing
572         for i, h in enumerate(ctx):
573             if h == phi:
574                 return ('hyp', i)
575         for nm, f in self.store.items():
576             if f == phi:
577                 return ('lemma', nm)
578         if depth == 0:
579             return None
580
581         k = phi[0]
582
583         if k == 'imp':
584             c = self.prove(phi[2], depth - 1, ctx + (phi[1],))
585             return ('impI', c) if c else None
586
587         if k == 'and':
588             c1 = self.prove(phi[1], depth - 1, ctx)
589             if not c1:
590                 return None
591             c2 = self.prove(phi[2], depth - 1, ctx)
592             return ('andI', c1, c2) if c2 else None
593
594         if k == 'exb':
595             n = teval(phi[2])
596             if n is None:

```

```

597         return None
598     for i in range(min(n, self.w)):
599         c = self.prove(sub(phi[3], phi[1], ('n', i)), depth - 1, ctx)
600         if c:
601             return ('exbI', i, c)
602     return None
603
604     if k == 'allb':
605         n = teval(phi[2])
606         if n is None or n > self.w:
607             return None
608         cs = []
609         for i in range(n):
610             c = self.prove(sub(phi[3], phi[1], ('n', i)), depth - 1, ctx)
611             if not c:
612                 return None
613             cs.append(c)
614         return ('allbI', cs)
615
616     if k == 'ex':
617         for t in self.witnesses(phi, ctx):
618             c = self.prove(sub(phi[2], phi[1], t), depth - 1, ctx)
619             if c:
620                 return ('wit', t, c)
621         return None
622
623     if k == 'all':
624         # (a) generalisation over a fresh eigenconstant. The name is
625         # recorded in the certificate; the kernel checks freshness.
626         nm = "e%d" % next(_eigen)
627         c = self.prove(sub(phi[2], phi[1], ('c', nm)), depth - 1, ctx)
628         if c:
629             return ('gen', nm, c)
630         # (b) induction
631         x, psi = phi[1], phi[2]
632         c0 = self.prove(sub(psi, x, ('n', 0)), depth - 1, ctx)
633         if c0:
634             step = ('all', x, ('imp', psi, sub(psi, x, ('s', ('v', x)))))
635             cs = self.prove(step, depth - 1, ctx)
636             if cs:
637                 return ('ind', c0, cs)
638         return None
639
640     return None
641
642 def witnesses(self, phi, ctx):
643     """Candidate witness terms for Ex.
644
645     w is exactly this list's length knob. Numerals alone cannot witness
646     a  $\Pi$ -0-2 goal --  $\forall x \exists y. x < y$  needs a term MENTIONING x -- so the
647     symbols already in scope are offered too. That is what lifts the
648     prover from Sigma-0-1 to tier 2."""
649     out = [('n', n) for n in range(self.w + 1)]
650     inscope = sorted(syms(phi) | {s for h in ctx for s in syms(h)})
651     for s in inscope:
652         base = ('c', s)
653         out.append(base)
654         t = base
655         for _ in range(min(self.w, 3)):
656             t = ('s', t)
657             out.append(t)
658         out.append(('*', ('n', 2), base))
659     return out
660
661 # =====
662 # REFLECTION LADDER
663 # =====
664 TAG = "<Ascent>"
665
666
667
668 def theta(k):
669     return ('atp', TAG) if k == 0 else ('pr', TAG, "<t%d>" % (k - 1))

```

```

670
671
672 def climb(kernel, store, names, r, verbose=True):
673     """Climb r rungs. Each rung is a kernel call on the previous
674     certificate -- Proposition 1 in code."""
675     certs, level, rungs = {}, 0, []
676     # theta_0 is the base case and is a stipulation, as Definition (Level-1)
677     # makes it. Everything above it is earned.
678     certs[0] = ('lemma', 't0')
679     store['t0'] = theta(0)
680     names["<t0>"] = theta(0)
681
682     for k in range(r):
683         phi = theta(k)
684         names.setdefault("<t%d>" % k, phi)
685         C = certs[k]
686         t0 = time.perf_counter()
687         try:
688             kernel.check(store, [], phi, C)
689         except Reject as e:
690             if verbose:
691                 print(" theta_%-2d REJECTED: %s" % (k, e))
692             break
693         level = k + 1
694         dt = time.perf_counter() - t0
695         rungs.append(dict(k=k, formula=show(phi), level=level,
696                         kernel_calls=kernel.calls, seconds=round(dt, 5)))
697         if verbose:
698             print(" theta_%-2d %-28s certified -> Level %d"
699                   % (k, show(phi), level))
700         # certified necessitation: the kernel accepted C : theta_k, so
701         # nec(C) is a certificate of theta_{k+1}.
702         nxt = theta(k + 1)
703         names["<t%d>" % (k + 1)] = nxt
704         certs[k + 1] = ('nec', C)
705         store["t%d" % (k + 1)] = nxt
706     return level, rungs
707
708
709 # =====
710 # TARGET SUITE
711 # =====
712 def X(n):
713     return ('v', n)
714
715
716 TARGETS = [
717     ("add_closed", ('=', ('+', ('n', 2), ('n', 2)), ('n', 4))),
718     ("bnd_lt", ('allb', 'x', ('n', 5), ('<', X('x'), ('n', 5)))),
719     ("bnd_ex", ('exb', 'x', ('n', 9), ('=', ('*', X('x'), X('x')),
720                                             ('n', 16)))),
721     ("ex_solve", ('ex', 'x', ('=', ('+', X('x'), ('n', 3)), ('n', 7))),
722     ("ex_square", ('ex', 'x', ('=', ('*', X('x'), X('x')), ('n', 49)))),
723     ("ex_big", ('ex', 'x', ('=', ('+', X('x'), ('n', 1)), ('n', 18))),
724     ("all_succ", ('all', 'x', ('<', X('x'), ('s', X('x'))))),
725     ("all_le", ('all', 'x', ('<=', X('x'), X('x')))),
726     ("all_add0", ('all', 'x', ('=', ('+', X('x'), ('n', 0)), X('x')))),
727     ("unbounded", ('all', 'x', ('ex', 'y', ('<', X('x'), X('y'))))),
728     ("succ_exists", ('all', 'x', ('ex', 'y', ('=', X('y'),
729                                             ('s', X('x'))))),
730     ("min_exists", ('ex', 'x', ('all', 'y', ('<=', X('x'), X('y'))))),
731 ]
732
733
734 def run_suite(d, w, verbose=True):
735     kernel = Kernel(TAG, {})
736     store = {}
737     S = Search(kernel, store, d, w)
738     rows, solved = [], 0
739     for name, phi in TARGETS:
740         t0 = time.perf_counter()
741         S.nodes = 0
742         C = S.prove(phi)

```

```

743     dt = time.perf_counter() - t0
744     verdict, ok = "", False
745     if C is not None:
746         try:
747             kernel.check(store, [], phi, C)
748             ok, verdict = True, "ok"
749             store[name] = phi
750         except Reject as e:
751             verdict = "REJECTED: %s" % e
752     solved += ok
753     rows.append(dict(target=name, tier=tier_name(phi),
754                     formula=show(phi), proved=C is not None,
755                     certified=ok, verdict=verdict,
756                     nodes=S.nodes, seconds=round(dt, 4)))
757     if verbose:
758         print(" %-12s %-7s %-34s %s"
759               % (name, tier_name(phi), show(phi)[:34],
760                 ("CERT ok, %d nodes" % S.nodes) if ok else
761                 (verdict or "not proved")))
762     return solved, rows, kernel
763
764
765 def tier_profile(rows):
766     prof = defaultdict(lambda: [0, 0])
767     for r in rows:
768         prof[r["tier"]][1] += 1
769         if r["certified"]:
770             prof[r["tier"]][0] += 1
771     return {k: dict(certified=v[0], targets=v[1])
772            for k, v in sorted(prof.items())}
773
774
775 # =====
776 def cmd_run(a):
777     print("=" * 74)
778     print(" ASCENT d=%d (depth) w=%d (witness) r=%d (reflect)"
779           % (a.depth, a.witness, a.reflect))
780     print("=" * 74)
781     print("\n arithmetic targets\n")
782     solved, rows, kernel = run_suite(a.depth, a.witness)
783
784     print("\n reflection ladder\n")
785     names = {}
786     store2 = {}
787     K2 = Kernel(TAG, names)
788     level, rungs = climb(K2, store2, names, a.reflect)
789
790     prof = tier_profile(rows)
791     highest = max([r["tier"] for r in rows if r["certified"]],
792                  default="none", key=lambda s: (s != "none", s))
793     tiers_hit = sorted([r["tier"] for r in rows if r["certified"]])
794
795     print("\n" + "-" * 74)
796     print(" certified %d/%d targets | tiers reached: %s"
797           % (solved, len(rows), ", ".join(tiers_hit) or "none"))
798     print(" lev(A) = %d (each rung kernel-certified; r is the only cap)"
799           % level)
800     print(" kernel calls: %d suite + %d ladder"
801           % (kernel.calls, K2.calls))
802     print("-" * 74)
803
804     os.makedirs(a.out, exist_ok=True)
805     with open(os.path.join(a.out, "ascent_d%d_w%d_r%d.json"
806                           % (a.depth, a.witness, a.reflect)), "w") as f:
807         json.dump(dict(params=dict(d=a.depth, w=a.witness, r=a.reflect),
808                         certified=solved, targets=len(rows),
809                         tier_profile=prof, tiers_reached=tiers_hit,
810                         lev=level, rows=rows, rungs=rungs), f, indent=2)
811     return 0
812
813
814 def cmd_grid(a):
815     """Proposition 2, empirically: sweep the knobs and show what moves."""

```

```

816 print("=" * 74)
817 print(" PARAMETER SWEEP -- what each knob buys")
818 print("=" * 74)
819 print("\n %-4s %-4s %-4s %-10s %-8s %-22s %s"
820       % ("d", "w", "r", "certified", "lev", "tiers reached", "nodes"))
821 print(" " + "-" * 70)
822 out = []
823 for d in a.depths:
824     for w in a.witnesses:
825         for r in a.reflects:
826             solved, rows, kernel = run_suite(d, w, verbose=False)
827             names, store2 = {}, {}
828             K2 = Kernel(TAG, names)
829             level, _ = climb(K2, store2, names, r, verbose=False)
830             tiers = sorted({x["tier"] for x in rows if x["certified"]})
831             nodes = sum(x["nodes"] for x in rows)
832             print(" %-4d %-4d %-4d %-10s %-8d %-22s %s"
833                   % (d, w, r, "%d/%d" % (solved, len(rows)), level,
834                     ", ".join(tiers) or "-", f"{nodes:}")")
835             out.append(dict(d=d, w=w, r=r, certified=solved,
836                             targets=len(rows), lev=level,
837                             tiers=tiers, nodes=nodes))
838 os.makedirs(a.out, exist_ok=True)
839 with open(os.path.join(a.out, "ascent_grid.json"), "w") as f:
840     json.dump(out, f, indent=2)
841
842 lev_by_r = defaultdict(set)
843 cert_by_dw = defaultdict(set)
844 for o in out:
845     lev_by_r[o["r"]].add(o["lev"])
846     cert_by_dw[(o["d"], o["w"])] .add(o["certified"])
847 ind_r = all(len(v) == 1 for v in lev_by_r.values())
848 ind_dw = all(len(v) == 1 for v in cert_by_dw.values())
849 print("\n lev depends only on r : %s" % ("YES" if ind_r else "NO"))
850 print(" certified count depends only on (d,w) : %s"
851       % ("YES" if ind_dw else "NO"))
852 print(" -> Proposition 2 holds on this grid" if (ind_r and ind_dw)
853       else " -> Proposition 2 FAILS on this grid")
854 return 0
855
856
857 def cmd_soundness(a):
858     """Hand the kernel certificates that must be rejected."""
859     print("=" * 74)
860     print(" KERNEL SOUNDNESS PROBES -- each of these MUST be rejected")
861     print("=" * 74 + "\n")
862     names = {"<f>": ('=', ('n', 0), ('n', 1))}
863     K = Kernel(TAG, names)
864     store = {}
865     bad = [
866         ("false by calc", ('=', ('n', 0), ('n', 1)), ('calc',)),
867         ("lemma that is not stored",
868          ('=', ('n', 0), ('n', 1)), ('lemma', 'nope')),
869         ("nec for an unproved formula",
870          ('pr', TAG, "<f>"), ('nec', ('calc',))),
871         ("nec with the wrong tag",
872          ('pr', "<Other>", "<f>"), ('nec', ('calc',))),
873         ("gen from a single instance",
874          ('all', 'x', ('=', X('x')), ('n', 0))),
875         ("gen", 'z', ('calc',))),
876         ("gen with a captured eigenvariable",
877          ('all', 'x', ('=', X('x')), ('c', 'k'))),
878         ("gen", 'k', ('calc',))),
879         ("wit with no witness",
880          ('ex', 'x', ('<', ('s', X('x')), X('x'))), ('wit', ('n', 3),
881                                                         ('calc',))),
882         ("induction without a base",
883          ('all', 'x', ('<', X('x')), ('n', 0))),
884         ("ind", ('calc',), ('calc',))),
885     ]
886     caught = 0
887     for label, phi, C in bad:
888         try:

```

```

889         K.check(store, [], phi, C)
890         print(" %-34s ACCEPTED <-- UNSOUND" % label)
891     except Reject as e:
892         caught += 1
893         print(" %-34s rejected (%s)" % (label, str(e)[:30]))
894     print("\n %d/%d rejected%s" % (caught, len(bad),
895                                   " " if caught == len(bad)
896                                   else " <-- KERNEL IS UNSOUND"))
897     return 0 if caught == len(bad) else 1
898
899
900 def main():
901     ap = argparse.ArgumentParser(
902         description=__doc__,
903         formatter_class=argparse.RawDescriptionHelpFormatter)
904     ap.add_argument("-d", "--depth", type=int, default=3)
905     ap.add_argument("-w", "--witness", type=int, default=10)
906     ap.add_argument("-r", "--reflect", type=int, default=4)
907     ap.add_argument("--out", default="results_ascent")
908     ap.add_argument("--grid", action="store_true")
909     ap.add_argument("--soundness", action="store_true")
910     ap.add_argument("--depths", type=int, nargs="+", default=[1, 2, 3, 4])
911     ap.add_argument("--witnesses", type=int, nargs="+", default=[1, 5, 20])
912     ap.add_argument("--reflects", type=int, nargs="+", default=[1, 3, 8])
913     a = ap.parse_args()
914     if a.soundness:
915         return cmd_soundness(a)
916     if a.grid:
917         return cmd_grid(a)
918     return cmd_run(a)
919
920
921 if __name__ == "__main__":
922     sys.exit(main())

```